

31-auth-identity

31. Authentication & Identity

Level: □ Intermediate **Jam:** 8 (4 minggu × 2 sesi)
Prasyarat: JavaScript Fundamentals, Node.js & Express, Database, REST API **Output:** Auth system — password, social login, SSO, MFA, security hardening

Tujuan Pembelajaran

Setelah modul ini, kamu bisa:

- Menerapkan password-based auth dengan bcrypt/argon2 + JWT
- Memahami OAuth2 flows (authorization code, PKCE, implicit)
- Integrasi social login (Google, GitHub) pakai Passport.js
- Bedain session vs JWT, dan pake refresh token rotation
- Bangun SSO + SAML vs OIDC
- Implementasi MFA (TOTP, SMS, authenticator app) & WebAuthn passkeys
- Magic Link / passwordless authentication
- Rate limiting, brute force protection, CSRF, secure cookies
- RBAC (role-based access control) & audit logging

Materi

Sesi	Topik	File
1	Password Auth — hashing, JWT, refresh token rotation, session (Redis), login/logout, email verification, forgot password	01-password-auth.md

Sesi	Topik	File
2	OAuth 2.0 & OIDC — flows, PKCE, social login Google/GitHub, Passport.js	02-oauth-oidc.md
3	SSO, SAML vs OIDC, Magic Link, MFA (TOTP/SMS/authenticator), WebAuthn passkeys	03-sso-mfa.md
4	Security Best Practices — rate limiting, brute force, CSRF, secure cookies, RBAC, audit log	04-security-best-practices.md

Output Akhir Modul

Auth System — REST API autentikasi lengkap: register/login password, social login (Google + GitHub), refresh token rotation, MFA (TOTP), SSO, role-based access control, rate limiting, audit log. Siap dipasang di aplikasi Express/TypeScript mana pun.

AI Prompt Exercises

Sepanjang modul, latihan pake AI:

- “Explain this JWT flow step by step”
- “Find security vulnerability in this auth middleware”
- “Refactor this OAuth2 callback to use PKCE”
- “Generate rate limiting config for 100 req/min per user”
- “Compare bcrypt vs argon2 for password hashing — which one should I use?”
- “Write a test case for refresh token rotation”
- “Explain OAuth2 authorization code flow like I’m 5”

1.1 Password-Based Authentication

Password Hashing

Jangan pernah simpan password mentah. Hash pake bcrypt atau argon2.

```

// src/utils/hash.ts
import bcrypt from 'bcrypt';

const SALT_ROUNDS = 12;

export async function hashPassword(password: string): Promise<string> {
  return bcrypt.hash(password, SALT_ROUNDS);
}

export async function verifyPassword(password: string, hash: string): Promise<boolean> {
  return bcrypt.compare(password, hash);
}

// Alternatif: argon2
import * as argon2 from 'argon2';

export async function hashPasswordArgon2(password: string): Promise<string> {
  return argon2.hash(password, {
    type: argon2.argon2id,
    memoryCost: 19456,
    timeCost: 2,
    parallelism: 1,
  });
}

export async function verifyPasswordArgon2(password: string, hash: string): Promise<boolean> {
  return argon2.verify(hash, password);
}

```

JWT — Access + Refresh Token

```

// src/utils/jwt.ts
import jwt from 'jsonwebtoken';

const ACCESS_SECRET = process.env.JWT_ACCESS_SECRET!;
const REFRESH_SECRET = process.env.JWT_REFRESH_SECRET!;

interface TokenPayload {
  userId: string;
  email: string;
  role: string;
}

export function generateAccessToken(payload: TokenPayload): string {
  return jwt.sign(payload, ACCESS_SECRET, { expiresIn: '15m' });
}

```

```

export function generateRefreshToken(payload: TokenPayload): string {
  return jwt.sign(payload, REFRESH_SECRET, { expiresIn: '7d' });
}

export function verifyAccessToken(token: string): TokenPayload {
  return jwt.verify(token, ACCESS_SECRET) as TokenPayload;
}

export function verifyRefreshToken(token: string): TokenPayload {
  return jwt.verify(token, REFRESH_SECRET) as TokenPayload;
}

```

Refresh Token Rotation

Setiap kali refresh dipake, token lama di-revoke dan yang baru dikasih.
Mencegah replay attack.

```

// src/services/auth.ts
import { v4 as uuidv4 } from 'uuid';
import { redis } from '../lib/redis';

interface RefreshTokenRecord {
  tokenId: string;
  userId: string;
  expiresAt: number;
}

export async function storeRefreshToken(userId: string, ttlSec: number = 7 * 24 * 60 * 60) {
  const tokenId = uuidv4();
  const record: RefreshTokenRecord = {
    tokenId,
    userId,
    expiresAt: Date.now() + ttlSec * 1000,
  };
  await redis.set(`refresh:${tokenId}`, JSON.stringify(record), 'EX', ttlSec);
  return tokenId;
}

export async function rotateRefreshToken(oldTokenId: string, userId: string): Promise<string> {
  // Cek old token masih valid
  const oldData = await redis.get(`refresh:${oldTokenId}`);
  if (!oldData) return null;

  // Revoke old
  await redis.del(`refresh:${oldTokenId}`);
}

```

```

    // Generate baru
    const tokenPayload = { userId, email: '', role: 'user' };
    const accessToken = generateAccessToken(tokenPayload);
    const newTokenId = await storeRefreshToken(userId);

    return { accessToken, refreshToken: newTokenId };
}

export async function revokeRefreshToken(tokenId: string): Promise<void> {
    await redis.del(`refresh:${tokenId}`);
}

export async function revokeAllUserTokens(userId: string): Promise<void> {
    const keys = await redis.keys(`refresh:*`);
    for (const key of keys) {
        const data = await redis.get(key);
        if (data) {
            const record: RefreshTokenRecord = JSON.parse(data);
            if (record.userId === userId) {
                await redis.del(key);
            }
        }
    }
}

```

Session Management dengan Redis

```

// src/lib/redis.ts
import Redis from 'ioredis';

export const redis = new Redis({
    host: process.env.REDIS_HOST || 'localhost',
    port: Number(process.env.REDIS_PORT) || 6379,
});

// Session store untuk express-session
import session from 'express-session';
import RedisStore from 'connect-redis';

export const sessionMiddleware = session({
    store: new RedisStore({ client: redis }),
    secret: process.env.SESSION_SECRET!,
    resave: false,
    saveUninitialized: false,
});

```

```

    cookie: {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
      maxAge: 24 * 60 * 60 * 1000, // 24 jam
    },
  });

```

Register & Login Flow

```

// src/routes/auth.ts
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { hashPassword, verifyPassword } from '../utils/hash';
import { generateAccessToken, generateRefreshToken, verifyRefreshToken } from '../utils/auth';
import { storeRefreshToken, rotateRefreshToken, revokeRefreshToken } from '../utils/refreshToken';
import { prisma } from '../lib/prisma';
import { sendVerificationEmail } from '../services/email';

const router = Router();

const registerSchema = z.object({
  email: z.string().email(),
  password: z.string().min(8).max(128),
  name: z.string().min(2).max(100),
});

// POST /api/auth/register
router.post('/register', async (req: Request, res: Response) => {
  const data = registerSchema.parse(req.body);

  const existing = await prisma.user.findUnique({ where: { email: data.email } });
  if (existing) {
    return res.status(409).json({ error: 'Email already registered' });
  }

  const passwordHash = await hashPassword(data.password);
  const user = await prisma.user.create({
    data: {
      email: data.email,
      passwordHash,
      name: data.name,
      emailVerified: false,
    },
  });
});

```

```

    // Kirim email verifikasi
    const verificationToken = crypto.randomUUID();
    await redis.set(`verify:${verificationToken}`, user.id, 'EX', 3600);
    await sendVerificationEmail(user.email, verificationToken);

    res.status(201).json({ message: 'User created. Check email to verify.' });
});

const loginSchema = z.object({
  email: z.string().email(),
  password: z.string(),
});

// POST /api/auth/login
router.post('/login', async (req: Request, res: Response) => {
  const data = loginSchema.parse(req.body);

  const user = await prisma.user.findUnique({ where: { email: data.email } });
  if (!user || !user.passwordHash) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  if (!user.emailVerified) {
    return res.status(403).json({ error: 'Email not verified' });
  }

  const valid = await verifyPassword(data.password, user.passwordHash);
  if (!valid) {
    return res.status(401).json({ error: 'Invalid credentials' });
  }

  const payload = { userId: user.id, email: user.email, role: user.role };

  // Session-based (optional)
  req.session.userId = user.id;

  // JWT-based
  const accessToken = generateAccessToken(payload);
  const refreshTokenId = await storeRefreshToken(user.id);

  res.json({
    accessToken,
    refreshToken: refreshTokenId,
    user: { id: user.id, email: user.email, name: user.name, role: user.role },
  });
});

```

```

});

// POST /api/auth/refresh
router.post('/refresh', async (req: Request, res: Response) => {
  const { refreshToken } = req.body;
  if (!refreshToken) {
    return res.status(400).json({ error: 'Refresh token required' });
  }

  // Decode dulu buat dapet userId
  let payload;
  try {
    payload = verifyRefreshToken(refreshToken);
  } catch {
    return res.status(401).json({ error: 'Invalid refresh token' });
  }

  const result = await rotateRefreshToken(refreshToken, payload.userId);
  if (!result) {
    return res.status(401).json({ error: 'Refresh token revoked or expired' });
  }

  res.json(result);
});

// POST /api/auth/logout
router.post('/logout', async (req: Request, res: Response) => {
  const { refreshToken } = req.body;
  if (refreshToken) {
    await revokeRefreshToken(refreshToken);
  }

  // Hapus session
  req.session.destroy((err) => {
    if (err) return res.status(500).json({ error: 'Logout failed' });
    res.clearCookie('connect.sid');
    res.json({ message: 'Logged out' });
  });
});

```

Email Verification

```

// src/routes/verify.ts
import { Router, Request, Response } from 'express';
import { prisma } from '../../lib/prisma';

```



```

import { redis } from '../lib/redis';

const router = Router();

// GET /api/auth/verify?token=xxx
router.get('/verify', async (req: Request, res: Response) => {
  const { token } = req.query;
  if (!token || typeof token !== 'string') {
    return res.status(400).json({ error: 'Verification token required' });
  }

  const userId = await redis.get(`verify:${token}`);
  if (!userId) {
    return res.status(400).json({ error: 'Invalid or expired token' });
  }

  await prisma.user.update({
    where: { id: userId },
    data: { emailVerified: true },
  });

  await redis.del(`verify:${token}`);

  res.json({ message: 'Email verified successfully' });
});

```

Forgot Password / Reset Password

```

// src/routes/password.ts
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { hashPassword } from '../utils/hash';
import { prisma } from '../lib/prisma';
import { redis } from '../lib/redis';
import { sendResetEmail } from '../services/email';

const router = Router();

// POST /api/auth/forgot-password
router.post('/forgot-password', async (req: Request, res: Response) => {
  const { email } = z.object({ email: z.string().email() }).parse(req.body);

  const user = await prisma.user.findUnique({ where: { email } });
  // Jangan bocorin apakah email terdaftar – selalu return success
  if (!user) {

```

```

    return res.json({ message: 'If email exists, reset link sent' });
  }

  const resetToken = crypto.randomUUID();
  await redis.set(`reset:${resetToken}`, user.id, 'EX', 3600); // 1 jam
  await sendResetEmail(user.email, resetToken);

  res.json({ message: 'If email exists, reset link sent' });
});

// POST /api/auth/reset-password
router.post('/reset-password', async (req: Request, res: Response) => {
  const { token, password } = z
    .object({
      token: z.string().uuid(),
      password: z.string().min(8).max(128),
    })
    .parse(req.body);

  const userId = await redis.get(`reset:${token}`);
  if (!userId) {
    return res.status(400).json({ error: 'Invalid or expired token' });
  }

  const passwordHash = await hashPassword(password);
  await prisma.user.update({
    where: { id: userId },
    data: { passwordHash },
  });

  await redis.del(`reset:${token}`);

  // Revoke all sessions & refresh tokens biar logout dari semua device
  await redis.keys(`refresh:*`).then((keys) => {
    keys.forEach(async (key) => {
      const data = await redis.get(key);
      if (data && JSON.parse(data).userId === userId) {
        await redis.del(key);
      }
    });
  });

  res.json({ message: 'Password reset successfully' });
});

```

Middleware Auth Guard

```
// src/middleware/auth.ts
import { Request, Response, NextFunction } from 'express';
import { verifyAccessToken } from '../utils/jwt';

// Extend Request type
declare global {
  namespace Express {
    interface Request {
      user?: { userId: string; email: string; role: string };
    }
  }
}

// JWT Auth Middleware
export function authenticate(req: Request, res: Response, next: NextFunction) {
  const header = req.headers.authorization;
  if (!header?.startsWith('Bearer ')) {
    return res.status(401).json({ error: 'No token provided' });
  }

  const token = header.slice(7);
  try {
    const payload = verifyAccessToken(token);
    req.user = payload;
    next();
  } catch {
    return res.status(401).json({ error: 'Invalid or expired token' });
  }
}

// Session Auth Middleware (alternatif)
export function authenticateSession(req: Request, res: Response, next: NextFunction) {
  if (!req.session.userId) {
    return res.status(401).json({ error: 'Not authenticated' });
  }
  next();
}
```

Latihan

1. **Register dengan validasi** — Tambahin validasi password strength (min 1 uppercase, 1 number, 1 special char) di route register. Pake zod regex.

2. **Refresh token rotation dengan family tree** — Implementasi token family (parent-child tracking). Kalo refresh token lama dipake setelah di-rotate, revoke seluruh family (detect token theft).
3. **Rate limiter per endpoint auth** — Pake express-rate-limit atau ioredis buat rate limit: 5 percobaan login per menit per IP, 3 percobaan forgot password per jam per email.
4. **Email verification + resend** — Tambah route POST /api/auth/resend-verification. Kirim ulang email verifikasi dengan cooldown 60 detik. Simpan lastSent di Redis.

2.1 OAuth 2.0 & OpenID Connect

OAuth 2.0 Flows

OAuth 2.0 punya beberapa grant types (flows). Yang paling umum:

Flow	Use Case	Keamanan
Authorization Code	Server-side apps (React + Express)	Paling aman
PKCE (Proof Key for Code Exchange)	Mobile / SPA tanpa backend rahasia	Aman
Client Credentials	Server-to-server (machine-to-machine)	Token aja
Implicit (deprecated)	SPA lama — JANGAN PAKE	Tidak aman

Authorization Code Flow (Web App)

```
// Step 1: Redirect user ke OAuth provider
// GET https://accounts.google.com/o/oauth2/v2/auth
//   ?client_id=xxx
//   &redirect_uri=http://localhost:3000/api/auth/google/callback
//   &response_type=code
//   &scope=openid%20profile%20email
//   &state=random-state-string

// Step 2: Provider redirect balik ke callback dengan code
// Step 3: Tukar code dengan access token

// src/services/oauth.ts
```

```

interface TokenResponse {
  access_token: string;
  refresh_token?: string;
  id_token?: string;
  expires_in: number;
}

export async function exchangeCodeForToken(
  code: string,
  provider: 'google' | 'github',
  redirectUri: string
): Promise<TokenResponse> {
  const configs = {
    google: {
      tokenUrl: 'https://oauth2.googleapis.com/token',
      clientId: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
    },
    github: {
      tokenUrl: 'https://github.com/login/oauth/access_token',
      clientId: process.env.GITHUB_CLIENT_ID!,
      clientSecret: process.env.GITHUB_CLIENT_SECRET!,
    },
  };

  const config = configs[provider];
  const params = new URLSearchParams({
    code,
    client_id: config.clientId,
    client_secret: config.clientSecret,
    redirect_uri: redirectUri,
    grant_type: 'authorization_code',
  });

  const res = await fetch(config.tokenUrl, {
    method: 'POST',
    headers: {
      'Content-Type': 'application/x-www-form-urlencoded',
      Accept: 'application/json',
    },
    body: params,
  });

  return res.json();
}

```

PKCE Flow (SPA / Mobile)

```
// Klien (frontend) generate code verifier + challenge
// src/utils/pkce.ts
import crypto from 'crypto';

export function generateCodeVerifier(): string {
  return base64URLEncode(crypto.randomBytes(32));
}

export function generateCodeChallenge(verifier: string): string {
  const hash = crypto.createHash('sha256').update(verifier).digest();
  return base64URLEncode(hash);
}

function base64URLEncode(buffer: Buffer): string {
  return buffer
    .toString('base64')
    .replace(/\+/g, '-')
    .replace(/\//g, '_')
    .replace(/=+$/, '');
}

// Frontend:
// 1. const verifier = generateCodeVerifier();
// 2. localStorage.setItem('pkce_verifier', verifier);
// 3. const challenge = generateCodeChallenge(verifier);
// 4. Redirect ke provider dengan &code_challenge={challenge}&code_challenge_met
//
// Backend exchange:
// params.set('code_verifier', verifierFromLocalStorage);
```

Client Credentials Flow (Server-to-Server)

```
// src/services/client-credentials.ts
export async function getClientCredentialsToken(
  clientId: string,
  clientSecret: string,
  scope: string,
  tokenUrl: string
): Promise<TokenResponse> {
  const params = new URLSearchParams({
    grant_type: 'client_credentials',
    client_id: clientId,
    client_secret: clientSecret,
    scope,
  });
```

```

});

const res = await fetch(tokenUrl, {
  method: 'POST',
  headers: { 'Content-Type': 'application/x-www-form-urlencoded' },
  body: params,
});

return res.json();
}

// Contoh: service account Google API
// const token = await getClientCredentialsToken(
//   process.env.SERVICE_CLIENT_ID!,
//   process.env.SERVICE_CLIENT_SECRET!,
//   'https://www.googleapis.com/auth/cloud-platform',
//   'https://oauth2.googleapis.com/token'
// );

```

OpenID Connect (OIDC)

OIDC adalah lapisan identitas di atas OAuth 2.0. Bedanya: dapat id_token (JWT) yang isinya identitas user.

```

// src/utils/oidc.ts
import jwt from 'jsonwebtoken';
import { JWKSClient } from './jwks';

interface IdTokenPayload {
  sub: string; // Subject – user ID dari provider
  email: string;
  email_verified: boolean;
  name: string;
  picture?: string;
  iss: string; // Issuer
  aud: string; // Audience – client ID
  exp: number;
  iat: number;
  nonce?: string; // Anti-replay
}

export async function verifyIdToken(
  idToken: string,
  clientId: string,
  issuer: string
): Promise<IdTokenPayload> {

```

```

// Dapetin public key dari OIDC discovery endpoint
const jwksClient = new JWKSClient(issuer);
const decoded = jwt.decode(idToken, { complete: true });
if (!decoded || !decoded.header.kid) {
  throw new Error('Invalid JWT: no kid');
}

const publicKey = await jwksClient.getPublicKey(decoded.header.kid);
const payload = jwt.verify(idToken, publicKey, {
  algorithms: ['RS256'],
  issuer,
  audience: clientId,
}) as IdTokenPayload;

return payload;
}

```

OIDC Discovery

```

// src/utils/jwks.ts
// Fetch OpenID configuration dari .well-known endpoint
export async function getOIDCConfig(issuer: string): Promise<{
  authorization_endpoint: string;
  token_endpoint: string;
  userinfo_endpoint: string;
  jwks_uri: string;
  issuer: string;
}> {
  const wellKnown = `${issuer}/.well-known/openid-configuration`;
  const res = await fetch(wellKnown);
  return res.json();
}

export class JWKSClient {
  private issuer: string;
  private keysCache: Map<string, string> = new Map();

  constructor(issuer: string) {
    this.issuer = issuer;
  }

  async getPublicKey(kid: string): Promise<string> {
    if (this.keysCache.has(kid)) {
      return this.keysCache.get(kid)!;
    }
  }
}

```



```

const config = await getOIDCConfig(this.issuer);
const res = await fetch(config.jwks_uri);
const { keys } = await res.json();

const key = keys.find((k: any) => k.kid === kid);
if (!key) throw new Error(`Key not found: ${kid}`);

const publicKey = convertJWKToPEM(key);
this.keysCache.set(kid, publicKey);
return publicKey;
}
}

```

Social Login dengan Passport.js

```

// src/config/passport.ts
import passport from 'passport';
import { Strategy as GoogleStrategy } from 'passport-google-oauth20';
import { Strategy as GitHubStrategy } from 'passport-github2';
import { prisma } from '../lib/prisma';

// Google Strategy
passport.use(
  new GoogleStrategy(
    {
      clientID: process.env.GOOGLE_CLIENT_ID!,
      clientSecret: process.env.GOOGLE_CLIENT_SECRET!,
      callbackURL: '/api/auth/google/callback',
      scope: ['profile', 'email'],
    },
    async (_accessToken, _refreshToken, profile, done) => {
      try {
        let user = await prisma.user.findFirst({
          where: {
            OR: [
              { googleId: profile.id },
              { email: profile.emails?.[0]?.value },
            ],
          },
        });

        if (!user) {
          // Auto-register
          user = await prisma.user.create({

```

```

        data: {
          email: profile.emails?.[0]?.value || '',
          name: profile.displayName,
          googleId: profile.id,
          emailVerified: true, // Google udah verify
          avatar: profile.photos?.[0]?.value,
        },
      });
    } else if (!user.googleId) {
      // Link akun
      user = await prisma.user.update({
        where: { id: user.id },
        data: { googleId: profile.id },
      });
    }

    done(null, user);
  } catch (err) {
    done(err as Error);
  }
}
)
);

// GitHub Strategy
passport.use(
  new GitHubStrategy(
    {
      clientId: process.env.GITHUB_CLIENT_ID!,
      clientSecret: process.env.GITHUB_CLIENT_SECRET!,
      callbackURL: '/api/auth/github/callback',
      scope: ['user:email'],
    },
    async (_accessToken, _refreshToken, profile, done) => {
      try {
        // Dapetin email dari GitHub (profile.emails kadang kosong)
        const email =
          profile.emails?.[0]?.value || `${profile.username}@github.placeholder`;

        let user = await prisma.user.findFirst({
          where: {
            OR: [
              { githubId: profile.id },
              { email },
            ],
          },
        },
      );
    }
  )
);

```

```

    });

    if (!user) {
      user = await prisma.user.create({
        data: {
          email,
          name: profile.displayName || profile.username || '',
          githubId: profile.id,
          emailVerified: true,
          avatar: profile.photos?.[0]?.value,
        },
      });
    } else if (!user.githubId) {
      user = await prisma.user.update({
        where: { id: user.id },
        data: { githubId: profile.id },
      });
    }

    done(null, user);
  } catch (err) {
    done(err as Error);
  }
}
)
);

// Serialize user ke session
passport.serializeUser((user: any, done) => {
  done(null, user.id);
});

passport.deserializeUser(async (id: string, done) => {
  try {
    const user = await prisma.user.findUnique({ where: { id } });
    done(null, user);
  } catch (err) {
    done(err);
  }
});

export default passport;

// src/routes/oauth.ts
import { Router, Request, Response } from 'express';
import passport from 'passport';

```

```

import { generateAccessToken, generateRefreshToken } from '../utils/jwt';
import { storeRefreshToken } from '../services/auth';

const router = Router();

// Initiate Google OAuth
router.get('/google', passport.authenticate('google', { session: false }));

// Google callback
router.get(
  '/google/callback',
  passport.authenticate('google', {
    session: false,
    failureRedirect: '/login?error=google-auth-failed',
  }),
  async (req: Request, res: Response) => {
    const user = req.user as any;
    const payload = { userId: user.id, email: user.email, role: user.role };
    const accessToken = generateAccessToken(payload);
    const refreshTokenId = await storeRefreshToken(user.id);

    // Redirect ke frontend dengan token
    res.redirect(
      `${process.env.FRONTEND_URL}/auth/callback?access_token=${accessToken}&ref
    );
  }
);

// Initiate Github OAuth
router.get('/github', passport.authenticate('github', { session: false }));

// Github callback
router.get(
  '/github/callback',
  passport.authenticate('github', {
    session: false,
    failureRedirect: '/login?error=github-auth-failed',
  }),
  async (req: Request, res: Response) => {
    const user = req.user as any;
    const payload = { userId: user.id, email: user.email, role: user.role };
    const accessToken = generateAccessToken(payload);
    const refreshTokenId = await storeRefreshToken(user.id);

    res.redirect(
      `${process.env.FRONTEND_URL}/auth/callback?access_token=${accessToken}&ref

```

```

    });
  }
});

// Link akun sosial ke user yang sudah login
// POST /api/auth/link/google
router.post('/link/google', async (req: Request, res: Response) => {
  // Pake passport.authenticate dengan custom callback
  passport.authenticate('google', { session: false }, async (err: any, profile:
    if (err || !profile) {
      return res.status(400).json({ error: 'Failed to link Google account' });
    }

    const currentUser = req.user as any;
    await prisma.user.update({
      where: { id: currentUser.userId },
      data: { googleId: profile.id },
    });

    res.json({ message: 'Google account linked' });
  })(req, res);
});

```

OAuth dengan State & Nonce

```

// src/middleware/oauth-state.ts
import { Request, Response, NextFunction } from 'express';
import crypto from 'crypto';
import { redis } from '../lib/redis';

// Generate state untuk CSRF protection
export async function generateOAuthState(userId?: string): Promise<string> {
  const state = crypto.randomUUID();
  const payload = JSON.stringify({ state, userId, createdAt: Date.now() });
  await redis.set(`oauth:state:${state}`, payload, 'EX', 600); // 10 menit
  return state;
}

// Verify state saat callback
export async function verifyOAuthState(state: string): Promise<boolean> {
  const data = await redis.get(`oauth:state:${state}`);
  if (!data) return false;

  await redis.del(`oauth:state:${state}`); // One-time use
  return true;
}

```

```

}

// Nonce untuk OIDC (anti-replay)
export function generateNonce(): string {
  return crypto.randomUUID();
}

// Simpen nonce di session / Redis
export async function storeNonce(nonce: string): Promise<void> {
  await redis.set(`oauth:nonce:${nonce}`, '1', 'EX', 600);
}

export async function verifyNonce(nonce: string): Promise<boolean> {
  const exists = await redis.get(`oauth:nonce:${nonce}`);
  if (!exists) return false;
  await redis.del(`oauth:nonce:${nonce}`);
  return true;
}

```

Model User (Prisma)

```

// schema.prisma
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  passwordHash String?
  name        String?
  avatar      String?
  role        String    @default("user")

  googleId    String?   @unique
  githubId    String?   @unique

  emailVerified Boolean @default(false)
  totpSecret   String?   // Untuk MFA nanti

  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  @@index([email])
  @@index([googleId])
  @@index([githubId])
}

```

Latihan

1. **Google + GitHub login di satu project** — Setup Passport.js dengan kedua strategi. Buat route `/api/auth/google` dan `/api/auth/github`. Test callback redirect ke frontend.
2. **PKCE flow manual** — Tanpa passport, implementasi PKCE flow dari nol: generate verifier, challenge, redirect ke Google, exchange code, verifikasi id_token. Pake node-fetch atau axios.
3. **OIDC dengan provider kustom** — Integrasi OIDC dengan provider lain (Microsoft/Azure AD, Apple, atau Keycloak lokal). Pake discovery endpoint dan JWKS verification.
4. **Account linking** — Buat endpoint `POST /api/auth/link` biar user yang login bisa connect akun Google/GitHub ke akun yang udah ada. Handle duplicate email.

3.1 SSO, SAML, Magic Link & MFA

Single Sign-On (SSO) Concepts

SSO = sekali login, akses banyak aplikasi. Dua protokol utama:

Protokol	Tipe	Kelebihan	Kekurangan
SAML 2.0	XML-based, enterprise	Mature, banyak dukungan enterprise	Berat (XML), kompleks
OIDC	JWT-based, modern	Ringan, REST API friendly	Relatif baru

SAML Flow (Simplified)

User -> Service Provider (SP) -> redirect ke Identity Provider (IdP)
IdP login -> generate SAML Assertion -> POST ke SP
SP verify assertion -> buat session lokal

```
// src/services/saml.ts
import crypto from 'crypto';
import { parseString, Builder } from 'xml2js';

export function generateSAMLResponse(assertion: SamlAssertion): string {
  const builder = new Builder({ headless: true });
  const samlResponse = {
```

```

'samlp:Response': {
  $: {
    xmlns: 'urn:oasis:names:tc:SAML:2.0:protocol',
    'xmlns:saml': 'urn:oasis:names:tc:SAML:2.0:assertion',
    ID: `_${crypto.randomUUID()}`,
    Version: '2.0',
    IssueInstant: new Date().toISOString(),
    Destination: assertion.acsUrl,
  },
  'saml:Issuer': assertion.issuer,
  'samlp:Status': {
    'samlp:StatusCode': {
      $: { Value: 'urn:oasis:names:tc:SAML:2.0:status:Success' } },
  },
},
'saml:Assertion': {
  $: {
    ID: `_${crypto.randomUUID()}`,
    IssueInstant: new Date().toISOString(),
    Version: '2.0',
  },
  'saml:Issuer': assertion.issuer,
  'saml:Subject': {
    'saml:NameID': {
      $: { Format: 'urn:oasis:names:tc:SAML:1.1:nameid-format:emailAddress' },
      _: assertion.email,
    },
    'saml:SubjectConfirmation': {
      $: { Method: 'urn:oasis:names:tc:SAML:2.0:cm:bearer' },
      'saml:SubjectConfirmationData': {
        $: {
          NotOnOrAfter: new Date(Date.now() + 5 * 60 * 1000).toISOString(),
          Recipient: assertion.acsUrl,
        },
      },
    },
  },
},
'saml:Conditions': {
  $: {
    NotBefore: new Date().toISOString(),
    NotOnOrAfter: new Date(Date.now() + 5 * 60 * 1000).toISOString(),
  },
  'saml:AudienceRestriction': {
    'saml:Audience': assertion.audience,
  },
},
},

```



```

'saml:AuthnStatement': {
  $: {
    AuthnInstant: new Date().toISOString(),
    SessionIndex: `_${crypto.randomUUID()}`,
  },
  'saml:AuthnContext': {
    'saml:AuthnContextClassRef': 'urn:oasis:names:tc:SAML:2.0:ac:classes
  },
},
'saml:AttributeStatement': {
  'saml:Attribute': [
    {
      $: { Name: 'email' },
      'saml:AttributeValue': assertion.email,
    },
    {
      $: { Name: 'name' },
      'saml:AttributeValue': assertion.name,
    },
    {
      $: { Name: 'role' },
      'saml:AttributeValue': assertion.role,
    },
  ],
},
},
};

return builder.buildObject(samlResponse);
}

// Parse SAML Response dari IdP
export function parseSAMLResponse(xml: string): Promise<SamlAssertion> {
  return new Promise((resolve, reject) => {
    parseString(xml, (err: any, result: any) => {
      if (err) return reject(err);
      try {
        const assertion = result['samlp:Response']['saml:Assertion'][0];
        const subject = assertion['saml:Subject'][0]['saml:NameID'][0]._;
        const attributes = assertion['saml:AttributeStatement'][0]['saml:Attribu
        const attrMap: Record<string, string> = {};
        attributes.forEach((attr: any) => {
          attrMap[attr.$.Name] = attr['saml:AttributeValue'][0];
        });
      }
    }
  });
}

```

```

        resolve({
            email: subject,
            name: attrMap['name'],
            role: attrMap['role'] || 'user',
            issuer: assertion['saml:Issuer'][0],
            audience: assertion['saml:Conditions'][0]['saml:AudienceRestriction'],
            acsUrl: '',
        });
    } catch (e) {
        reject(e);
    }
});
});
}
}

interface SamlAssertion {
    email: string;
    name: string;
    role: string;
    issuer: string;
    audience: string;
    acsUrl: string;
}

```

SAML vs OIDC — Kapan Pake?

- **Pilih SAML** kalo: enterprise, integrasi dengan Active Directory / ADFS / Okta legacy, butuh attribute-based access yang kompleks
- **Pilih OIDC** kalo: modern web/mobile apps, social login, REST API, mikroservices

Magic Link (Passwordless)

```

// src/services/magic-link.ts
import crypto from 'crypto';
import { redis } from '../lib/redis';
import { prisma } from '../lib/prisma';
import { generateAccessToken, generateRefreshToken } from '../utils/jwt';
import { storeRefreshToken } from '../auth';

const MAGIC_LINK_TTL = 15 * 60; // 15 menit

export async function sendMagicLink(email: string): Promise<void> {
    const token = crypto.randomBytes(32).toString('hex');
    const hash = crypto.createHash('sha256').update(token).digest('hex');
}

```

```

// Simpan hash di Redis
await redis.set(`magic:${hash}`, email, 'EX', MAGIC_LINK_TTL);

// Kirim email
const link = `${process.env.FRONTEND_URL}/auth/magic?token=${token}`;
// await sendEmail(email, 'Your magic sign-in link', `Click: ${link}`);
console.log(`Magic link for ${email}: ${link}`);
}

export async function verifyMagicLink(token: string): Promise<{
  accessToken: string;
  refreshToken: string;
  user: any;
} | null> {
  const hash = crypto.createHash('sha256').update(token).digest('hex');
  const email = await redis.get(`magic:${hash}`);

  if (!email) return null;

  // One-time use
  await redis.del(`magic:${hash}`);

  // Cari atau buat user
  let user = await prisma.user.findUnique({ where: { email } });
  if (!user) {
    user = await prisma.user.create({
      data: {
        email,
        name: email.split('@')[0],
        emailVerified: true,
      },
    });
  }

  const payload = { userId: user.id, email: user.email, role: user.role };
  const accessToken = generateAccessToken(payload);
  const refreshTokenId = await storeRefreshToken(user.id);

  return { accessToken, refreshToken: refreshTokenId, user };
}

// src/routes/magic-link.ts
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { sendMagicLink, verifyMagicLink } from '../services/magic-link';

```

```

const router = Router();

// POST /api/auth/magic-link - kirim link
router.post('/magic-link', async (req: Request, res: Response) => {
  const { email } = z.object({ email: z.string().email() }).parse(req.body);
  await sendMagicLink(email);
  res.json({ message: 'Magic link sent if email exists' });
});

// GET /api/auth/magic-link/verify?token=xxx
router.get('/magic-link/verify', async (req: Request, res: Response) => {
  const { token } = z.object({ token: z.string() }).parse(req.query);
  const result = await verifyMagicLink(token);

  if (!result) {
    return res.status(400).json({ error: 'Invalid or expired token' });
  }

  res.json(result);
});

```

Multi-Factor Authentication (MFA)

TOTP (Time-based One-Time Password)

```

// src/services/totp.ts
import * as speakeasy from 'speakeasy';
import * as qrcode from 'qrcode';

export interface TOTPSetup {
  secret: string;
  qrCodeUrl: string;
}

// Setup TOTP untuk user
export function setupTOTP(email: string): TOTPSetup {
  const secret = speakeasy.generateSecret({
    name: `RPLApp:${email}`,
    issuer: 'RPLApp',
  });

  return {
    secret: secret.base32,
    qrCodeUrl: secret.otppath_url || '',
  };
}

```

```

}

// Generate QR code data URL
export async function generateQRCode(otpauthUrl: string): Promise<string> {
  return qrcode.toDataURL(otpauthUrl);
}

// Verify TOTP code
export function verifyTOTP(token: string, secret: string): boolean {
  return speakeasy.totp.verify({
    secret,
    encoding: 'base32',
    token,
    window: 1, // ±30 detik toleransi
  });
}

// Generate QR code langsung
export async function getTOTPSetup(email: string): Promise<{
  secret: string;
  qrCode: string; // data URL
}> {
  const { secret, qrCodeUrl } = setupTOTP(email);
  const qrCode = await generateQRCode(qrCodeUrl);
  return { secret, qrCode };
}

```

MFA — Enable & Verify Routes

```

// src/routes/mfa.ts
import { Router, Request, Response } from 'express';
import { z } from 'zod';
import { authenticate } from '../middleware/auth';
import { prisma } from '../lib/prisma';
import { getTOTPSetup, verifyTOTP } from '../services/totp';

const router = Router();

// GET /api/mfa/setup – dapetin QR code + secret
router.get('/setup', authenticate, async (req: Request, res: Response) => {
  const user = await prisma.user.findUnique({
    where: { id: req.user!.userId },
  });

  if (!user) return res.status(404).json({ error: 'User not found' });

```

```

    if (user.totpSecret) {
      return res.status(400).json({ error: 'MFA already enabled' });
    }

    const { secret, qrCode } = await getTOTPSetup(user.email);

    // Simpan secret sementara di session / Redis
    // Jangan simpan di DB sampai diverifikasi
    req.session.pendingTotpSecret = secret;

    res.json({ qrCode, secret });
  });

  // POST /api/mfa/verify – verifikasi + enable MFA
  router.post(
    '/verify',
    authenticate,
    async (req: Request, res: Response) => {
      const { token } = z.object({ token: z.string().length(6) }).parse(req.body);
      const secret = req.session.pendingTotpSecret;

      if (!secret) {
        return res.status(400).json({ error: 'No pending TOTP setup. Call /setup f' });
      }

      if (!verifyTOTP(token, secret)) {
        return res.status(400).json({ error: 'Invalid token' });
      }

      // Simpan secret di DB
      await prisma.user.update({
        where: { id: req.user!.userId },
        data: { totpSecret: secret },
      });

      delete req.session.pendingTotpSecret;

      res.json({ message: 'MFA enabled successfully' });
    }
  );

  // POST /api/mfa/disable – disable MFA
  router.post('/disable', authenticate, async (req: Request, res: Response) => {
    const { token } = z.object({ token: z.string().length(6) }).parse(req.body);
    const user = await prisma.user.findUnique({ where: { id: req.user!.userId } })

```

```

    if (!user?.totpSecret) {
      return res.status(400).json({ error: 'MFA not enabled' });
    }

    if (!verifyTOTP(token, user.totpSecret)) {
      return res.status(400).json({ error: 'Invalid token' });
    }

    await prisma.user.update({
      where: { id: user.id },
      data: { totpSecret: null },
    });

    res.json({ message: 'MFA disabled' });
  });

```

MFA — Login Flow

```

// src/middleware/mfa.ts
import { Request, Response, NextFunction } from 'express';

// Extended di login: setelah password/oauth valid, cek MFA
export async function requireMFA(req: Request, res: Response, next: NextFunction) {
  const user = await prisma.user.findUnique({ where: { id: req.user!.userId } });

  if (user?.totpSecret) {
    // Step 1: kirim token sementara (MFA challenge)
    const mfaToken = crypto.randomUUID();
    await redis.set(`mfa:${mfaToken}`, user.id, 'EX', 300); // 5 menit
    return res.json({
      requireMfa: true,
      mfaToken,
      // Frontend: redirect ke form input TOTP
    });
  }

  // No MFA — proceed normal
  next();
}

// POST /api/auth/mfa-challenge
router.post('/mfa-challenge', async (req: Request, res: Response) => {
  const { mfaToken, code } = z
    .object({
      mfaToken: z.string().uuid(),

```

```

        code: z.string().length(6),
      })
      .parse(req.body);

const userId = await redis.get(`mfa:${mfaToken}`);
if (!userId) return res.status(400).json({ error: 'MFA challenge expired' });

const user = await prisma.user.findUnique({ where: { id: userId } });
if (!user?.totpSecret) return res.status(400).json({ error: 'MFA not configured' });

if (!verifyTOTP(code, user.totpSecret)) {
  return res.status(401).json({ error: 'Invalid MFA code' });
}

// MFA passed – generate final tokens
await redis.del(`mfa:${mfaToken}`);

const payload = { userId: user.id, email: user.email, role: user.role };
const accessToken = generateAccessToken(payload);
const refreshTokenId = await storeRefreshToken(user.id);

res.json({ accessToken, refreshToken: refreshTokenId, user });
});

```

SMS-based MFA (via Twilio)

```

// src/services/sms-mfa.ts
import twilio from 'twilio';

const client = twilio(process.env.TWILIO_SID!, process.env.TWILIO_AUTH_TOKEN!);

// Generate & kirim SMS
export async function sendSmsCode(phoneNumber: string): Promise<string> {
  const code = Math.floor(100000 + Math.random() * 900000).toString();
  const hash = crypto.createHash('sha256').update(code).digest('hex');

  await redis.set(`sms:${hash}`, phoneNumber, 'EX', 300); // 5 menit

  await client.messages.create({
    body: `Your verification code: ${code}`,
    from: process.env.TWILIO_PHONE!,
    to: phoneNumber,
  });

  return hash; // Jangan kirim code ke response!
}

```



```

}

// Verifikasi
export async function verifySmsCode(hash: string, code: string): Promise<boolean> {
  const expectedHash = crypto.createHash('sha256').update(code).digest('hex');
  if (hash !== expectedHash) return false;

  const exists = await redis.get(`sms:${hash}`);
  if (!exists) return false;

  await redis.del(`sms:${hash}`);
  return true;
}

```

WebAuthn — Passkeys

WebAuthn (FIDO2) = login tanpa password pake biometric / hardware key.

```

// src/services/webauthn.ts
import { generateAuthenticationOptions, generateRegistrationOptions, verifyAuthn } from '@simplewebauthn/server';
import { isoBase64URL } from '@simplewebauthn/server/helpers';
import { redis } from '../../lib/redis';

const RP_NAME = 'RPL App';
const RP_ID = process.env.DOMAIN || 'localhost';
const ORIGIN = process.env.ORIGIN || 'http://localhost:3000';

// 1. Generate registration options (untuk create passkey)
export async function generateWebAuthnRegistrationOptions(userId: string, email: string) {
  const options = await generateRegistrationOptions({
    rpName: RP_NAME,
    rpID: RP_ID,
    userName: email,
    userID: isoBase64URL.fromString(userId),
    attestationType: 'none',
    authenticatorSelection: {
      residentKey: 'preferred',
      userVerification: 'preferred',
    },
  });

  // Simpan challenge di session
  await redis.set(`webauthn:challenge:${userId}`, options.challenge, 'EX', 300);

  return options;
}

```

```

}

// 2. Verify registration response (simpan credential)
export async function verifyWebAuthnRegistration(
  userId: string,
  credential: any
) {
  const challenge = await redis.get(`webauthn:challenge:${userId}`);
  if (!challenge) throw new Error('Challenge not found');

  const verification = await verifyRegistrationResponse({
    response: credential,
    expectedChallenge: challenge,
    expectedOrigin: ORIGIN,
    expectedRPID: RP_ID,
  });

  if (verification.verified && verification.registrationInfo) {
    const { credentialPublicKey, credentialID, counter } = verification.registrationInfo;

    // Simpan ke DB
    // await prisma.credential.create({
    //   data: {
    //     userId,
    //     credentialId: isoBase64URL.fromBuffer(credentialID),
    //     publicKey: isoBase64URL.fromBuffer(credentialPublicKey),
    //     counter,
    //   },
    // });
  }

  await redis.del(`webauthn:challenge:${userId}`);
  return verification.verified;
}

// 3. Generate authentication options (untuk login)
export async function generateWebAuthnAuthOptions(userId: string, credentials: any[]) {
  const options = await generateAuthenticationOptions({
    rpID: RP_ID,
    allowCredentials: credentials.map((cred) => ({
      id: cred.credentialId,
      type: 'public-key',
    })),
    userVerification: 'preferred',
  });
}

```

```

    await redis.set(`webauthn:challenge:${userId}`, options.challenge, 'EX', 300);
    return options;
}

// 4. Verify authentication response
export async function verifyWebAuthnAuth(userId: string, credential: any) {
    const challenge = await redis.get(`webauthn:challenge:${userId}`);
    if (!challenge) throw new Error('Challenge not found');

    // Ambil credential dari DB
    // const storedCred = await prisma.credential.findUnique({
    //   where: { id: credential.id },
    // });

    const verification = await verifyAuthenticationResponse({
        response: credential,
        expectedChallenge: challenge,
        expectedOrigin: ORIGIN,
        expectedRPID: RP_ID,
        // credential: {
        //   id: storedCred.credentialId,
        //   publicKey: isoBase64URL.toBuffer(storedCred.publicKey),
        //   counter: storedCred.counter,
        // },
    }) as any;

    if (verification.verified) {
        // Update counter
        // await prisma.credential.update({
        //   where: { id: credential.id },
        //   data: { counter: verification.authenticationInfo.newCounter },
        // });
    }

    await redis.del(`webauthn:challenge:${userId}`);
    return verification.verified;
}

```

Latihan

1. **Magic Link full flow** — Buat route register+login via magic link. Generate token, kirim ke email (console aja), verify, auto-create user kalo belum ada. Include rate limit: 1 link per 60 detik per email.
2. **TOTP MFA login** — Integrasi TOTP ke login flow: user login →

cek totpSecret → kalo ada, minta TOTP code → baru kasih JWT.
Pake speakeasy + qrcode library.

3. **SMS MFA via mock** — Tanpa Twilio beneran, bikin mock SMS service. Simulasi kirim kode 6 digit via console. Verifikasi kode pake hash comparison.
4. **WebAuthn passkey registration** — Setup endpoint registration WebAuthn: generate options → simpan challenge di Redis → verify response → simpan credential ke DB (Prisma model Credential).

4.1 Security Best Practices

Rate Limiting per User

```
// src/middleware/rate-limit.ts
import rateLimit from 'express-rate-limit';
import RedisStore from 'rate-limit-redis';
import { redis } from '../lib/redis';

// Global rate limit
export const globalRateLimit = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redis.call(...args),
  }),
  windowMs: 15 * 60 * 1000, // 15 menit
  max: 100,
  standardHeaders: true,
  legacyHeaders: false,
  message: { error: 'Too many requests, try again later' },
});

// Strict rate limit untuk auth endpoints
export const authRateLimit = rateLimit({
  store: new RedisStore({
    sendCommand: (...args: string[]) => redis.call(...args),
  }),
  windowMs: 60 * 1000, // 1 menit
  max: 5, // 5 percobaan per menit
  skipSuccessfulRequests: true, // Hitung cuma yang gagal
  message: { error: 'Too many login attempts, try again later' },
});

// Per-user rate limiter pake user ID (kalo udah login)
export async function checkUserRateLimit(
```

```

    userId: string,
    action: string,
    maxAttempts: number = 10,
    windowSec: number = 60
  ): Promise<boolean> {
    const key = `ratelimit:${action}:${userId}`;
    const current = await redis.incr(key);

    if (current === 1) {
      await redis.expire(key, windowSec);
    }

    return current <= maxAttempts;
  }

  // Middleware wrapper
  export function userRateLimit(action: string, max?: number, windowSec?: number)
  return async (req: Request, res: Response, next: NextFunction) => {
    const userId = req.user?.userId;
    if (!userId) return next();

    const allowed = await checkUserRateLimit(userId, action, max, windowSec);
    if (!allowed) {
      return res.status(429).json({ error: 'Rate limit exceeded per user' });
    }
    next();
  };
}

```

Brute Force Protection

```

// src/services/brute-force.ts
import { redis } from '../lib/redis';

const MAX_ATTEMPTS = 5;
const LOCKOUT_DURATION = 15 * 60; // 15 menit detik
const ATTEMPT_WINDOW = 60; // 1 menit

type IdentifierType = 'ip' | 'email' | 'userId';

export async function recordFailedAttempt(identifier: string, type: IdentifierType) {
  const key = `bruteforce:${type}:${identifier}`;
  const attempts = await redis.incr(key);

  if (attempts === 1) {

```

```

    await redis.expire(key, ATTEMPT_WINDOW);
  }

  if (attempts >= MAX_ATTEMPTS) {
    // Lockout
    const lockKey = `lockout:${type}:${identifier}`;
    await redis.set(lockKey, '1', 'EX', LOCKOUT_DURATION);
    await redis.del(key);
  }
}

export async function isLockedOut(identifier: string, type: IdentifierType): Promise<boolean> {
  const lockKey = `lockout:${type}:${identifier}`;
  const locked = await redis.get(lockKey);
  return !!locked;
}

export async function resetAttempts(identifier: string, type: IdentifierType): Promise<void> {
  const key = `bruteforce:${type}:${identifier}`;
  const lockKey = `lockout:${type}:${identifier}`;
  await redis.del(key);
  await redis.del(lockKey);
}

// Brute force middleware buat login
export async function bruteForceCheck(req: Request, res: Response, next: NextFunction) {
  const ip = req.ip || req.socket.remoteAddress || 'unknown';
  const email = req.body?.email;

  // Cek IP lockout
  if (await isLockedOut(ip, 'ip')) {
    return res.status(429).json({
      error: 'Account temporarily locked. Try again in 15 minutes.',
    });
  }

  // Cek email lockout (kalo ada)
  if (email && (await isLockedOut(email, 'email'))) {
    return res.status(429).json({
      error: 'Account temporarily locked. Try again in 15 minutes.',
    });
  }

  next();
}

```

```

// Record failed login – panggil setelah verifikasi gagal
export async function recordFailedLogin(email: string, req: Request): Promise<void> {
  const ip = req.ip || req.socket.remoteAddress || 'unknown';
  await recordFailedAttempt(ip, 'ip');
  await recordFailedAttempt(email, 'email');
}

```

CSRF Tokens

```

// src/middleware/csrf.ts
import csrf from 'csrf';
import { Request, Response, NextFunction } from 'express';

const tokens = new csrf();

// Pastikan ini dipasang setelah cookie-parser & session
export function csrfProtection(req: Request, res: Response, next: NextFunction) {
  // Bypass buat API yang pake JWT (Bearer token)
  const authHeader = req.headers.authorization;
  if (authHeader?.startsWith('Bearer ')) {
    return next();
  }

  // Bypass buat GET/HEAD/OPTIONS (idempotent)
  if (['GET', 'HEAD', 'OPTIONS'].includes(req.method)) {
    // Generate token kalo belum ada
    if (!req.session.csrfSecret) {
      req.session.csrfSecret = tokens.secretSync();
    }
    // Kirim token via cookie (dibaca frontend)
    const csrfToken = tokens.create(req.session.csrfSecret);
    res.cookie('csrf-token', csrfToken, {
      httpOnly: false, // Frontend perlu baca
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'strict',
    });
    return next();
  }

  // POST/PUT/DELETE: verifikasi token
  const secret = req.session.csrfSecret;
  if (!secret) {
    return res.status(403).json({ error: 'CSRF token missing' });
  }
}

```

```

const token = req.headers['x-csrf-token'] as string;
if (!token || !tokens.verify(secret, token)) {
  return res.status(403).json({ error: 'Invalid CSRF token' });
}

next();
}

// Atau pake library csurf (legacy) – alternatif:
// import csurf from 'csurf';
// export const csrfProtection = csurf({ cookie: true });

```

Secure Cookies

```

// src/config/cookie.ts
import { CookieOptions } from 'express';

export const secureCookieOptions: CookieOptions = {
  httpOnly: true, // Gak bisa diakses JavaScript
  secure: process.env.NODE_ENV === 'production', // HTTPS-only di prod
  sameSite: 'strict', // Gak dikirim dari site lain
  maxAge: 7 * 24 * 60 * 60 * 1000, // 7 hari
  path: '/',
  // domain: '.yourdomain.com', // Kalo butuh cross-subdomain
};

export const sessionCookieOptions: CookieOptions = {
  ...secureCookieOptions,
  maxAge: 24 * 60 * 60 * 1000, // 24 jam
};

// Set cookie di response
// res.cookie('refresh_token', refreshToken, secureCookieOptions);
// res.cookie('session_id', sessionId, sessionCookieOptions);

```

Cookie Flag Perbandingan

Flag	Fungsi
HttpOnly	Mencegah XSS — gak bisa dibaca JavaScript
Secure	Cuma dikirim via HTTPS
SameSite=Strict	Mencegah CSRF — gak dikirim dari origin lain
SameSite=Lax	CSRF protection + link bisa jalan
MaxAge	Expiry otomatis
Domain	Batasi ke domain tertentu

Flag	Fungsi
------	--------

Session Fixation Protection

```
// src/middleware/session-fixation.ts
import { Request, Response, NextFunction } from 'express';

// Regenerasi session ID setelah login biar attacker
// gak bisa pake session ID yang udah ditebak sebelumnya
export function regenerateSession(req: Request, res: Response, next: NextFunction) {
  if (req.session) {
    req.session.regenerate((err) => {
      if (err) return next(err);
      next();
    });
  } else {
    next();
  }
}

// Pake di route login:
// router.post('/login', regenerateSession, async (req, res) => { ... });
```

RBAC — Role-Based Access Control

```
// src/middleware/rbac.ts
import { Request, Response, NextFunction } from 'express';

// Role hierarchy
const ROLES = {
  admin: ['admin', 'moderator', 'user'],
  moderator: ['moderator', 'user'],
  user: ['user'],
} as const;

type Role = keyof typeof ROLES;

export function authorize(...allowedRoles: Role[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    const userRole = req.user?.role as Role;

    if (!userRole) {
      return res.status(401).json({ error: 'Not authenticated' });
    }
  }
}
```

```

    const hasAccess = allowedRoles.some((role) =>
      ROLES[role]?.includes(userRole)
    );

    if (!hasAccess) {
      return res.status(403).json({ error: 'Insufficient permissions' });
    }

    next();
  };
}

```

```

// Usage:
// router.get('/admin/users', authenticate, authorize('admin'), handler);
// router.patch('/posts/:id', authenticate, authorize('admin', 'moderator'), handler);

```

Resource-Level Authorization

```

// src/middleware/ownership.ts
import { Request, Response, NextFunction } from 'express';

// Cek kepemilikan resource
export function authorizeResource(
  getResourceOwnerId: (req: Request) => Promise<string | null>
) {
  return async (req: Request, res: Response, next: NextFunction) => {
    const userId = req.user?.userId;
    if (!userId) return res.status(401).json({ error: 'Not authenticated' });

    // Admin always pass
    if (req.user?.role === 'admin') return next();

    const ownerId = await getResourceOwnerId(req);
    if (!ownerId || ownerId !== userId) {
      return res.status(403).json({ error: 'Not authorized for this resource' });
    }

    next();
  };
}

// Usage:
// router.delete(
//   '/posts/:id',

```

```

//   authenticate,
//   authorizeResource(async (req) => {
//     const post = await prisma.post.findUnique({ where: { id: req.params.id }
//     return post?.authorId || null;
//   }),
//   handler
// );

```

Permission-Based Access (lebih granular dari RBAC)

```

// src/services/permissions.ts
// Define permissions
export const Permissions = {
  users: {
    read: 'users:read',
    create: 'users:create',
    update: 'users:update',
    delete: 'users:delete',
  },
  posts: {
    read: 'posts:read',
    create: 'posts:create',
    update: 'posts:update',
    delete: 'posts:delete',
  },
} as const;

// Role-permission mapping
const ROLE_PERMISSIONS: Record<string, string[]> = {
  admin: Object.values(Permissions).flatMap((p) => Object.values(p)),
  moderator: [
    Permissions.posts.read,
    Permissions.posts.create,
    Permissions.posts.update,
    Permissions.users.read,
  ],
  user: [
    Permissions.posts.read,
    Permissions.posts.create,
  ],
};

export function requirePermission(...permissions: string[]) {
  return (req: Request, res: Response, next: NextFunction) => {
    const role = req.user?.role || 'user';

```

```

    const userPermissions = ROLE_PERMISSIONS[role] || [];

    const hasAll = permissions.every((p) => userPermissions.includes(p));
    if (!hasAll) {
        return res.status(403).json({ error: 'Missing permission' });
    }

    next();
};
}

```

Audit Logging

```

// src/middleware/audit.ts
import { Request, Response, NextFunction } from 'express';
import { prisma } from '../lib/prisma';

// Audit log entry
interface AuditEntry {
    userId?: string;
    action: string; // 'user.login', 'user.logout', 'user.password_reset', '
    resource: string; // 'user', 'post', 'comment'
    resourceId?: string;
    metadata?: Record<string, any>;
    ip?: string;
    userAgent?: string;
    success: boolean;
}

// Log to database (via Prisma)
export async function logAudit(entry: AuditEntry): Promise<void> {
    await prisma.auditLog.create({
        data: {
            userId: entry.userId,
            action: entry.action,
            resource: entry.resource,
            resourceId: entry.resourceId,
            metadata: entry.metadata || {},
            ip: entry.ip,
            userAgent: entry.userAgent,
            success: entry.success,
        },
    });
}

```

```

// Audit middleware
export function audit(action: string, resource: string) {
  return async (req: Request, res: Response, next: NextFunction) => {
    const originalSend = res.send.bind(res);

    res.send = function (body: any): Response {
      const success = res.statusCode < 400;

      logAudit({
        userId: req.user?.userId,
        action,
        resource,
        resourceId: req.params?.id || req.body?.id,
        metadata: {
          method: req.method,
          path: req.path,
          body: sanitizeBody(req.body),
        },
        ip: req.ip,
        userAgent: req.headers['user-agent'],
        success,
      }).catch(console.error);

      return originalSend(body);
    };

    next();
  };
}

// Jangan log password
function sanitizeBody(body: any): Record<string, any> {
  if (!body) return {};
  const sanitized = { ...body };
  delete sanitized.password;
  delete sanitized.passwordHash;
  delete sanitized.token;
  delete sanitized.secret;
  return sanitized;
}

// Usage:
// router.post('/login', audit('user.login', 'user'), loginHandler);
// router.delete('/posts/:id', authenticate, audit('post.delete', 'post'), deleteHandler);

```

Audit Log Query

```
// src/services/audit-query.ts
// Admin panel - liat log
export async function getAuditLogs(options: {
  userId?: string;
  action?: string;
  resource?: string;
  from?: Date;
  to?: Date;
  page?: number;
  limit?: number;
}) {
  const { userId, action, resource, from, to, page = 1, limit = 50 } = options;

  const where: any = {};
  if (userId) where.userId = userId;
  if (action) where.action = action;
  if (resource) where.resource = resource;
  if (from || to) {
    where.createdAt = {};
    if (from) where.createdAt.gte = from;
    if (to) where.createdAt.lte = to;
  }

  const [logs, total] = await Promise.all([
    prisma.auditLog.findMany({
      where,
      orderBy: { createdAt: 'desc' },
      skip: (page - 1) * limit,
      take: limit,
      include: { user: { select: { email: true, name: true } } },
    }),
    prisma.auditLog.count({ where }),
  ]);

  return { logs, total, page, limit, pages: Math.ceil(total / limit) };
}
```

Prisma Model untuk Audit

```
model AuditLog {
  id          String    @id @default(cuid())
  userId      String?
  action      String    // 'user.login', 'post.delete', dll
  resource    String    // 'user', 'post'
```

```

resourceId String?
metadata   Json    @default("{}")
ip         String?
userAgent  String?
success    Boolean
createdAt  DateTime @default(now())

user       User?    @relation(fields: [userId], references: [id])

@@index([userId])
@@index([action])
@@index([resource])
@@index([createdAt])
@@index([action, createdAt])
}

```

Security Headers

```

// src/middleware/security-headers.ts
import helmet from 'helmet';
import { Request, Response, NextFunction } from 'express';

// Helmet sets these headers:
// - Content-Security-Policy
// - X-Content-Type-Options: nosniff
// - X-Frame-Options: DENY
// - X-XSS-Protection: 0
// - Strict-Transport-Security
// - Referrer-Policy

export const securityHeaders = helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc: ["'self'"],
      styleSrc: ["'self'", "'unsafe-inline'"],
      imgSrc: ["'self'", 'data:', 'https:'],
      connectSrc: ["'self'", process.env.API_URL || ''],
    },
  },
  hsts: {
    maxAge: 31536000, // 1 year
    includeSubDomains: true,
    preload: true,
  },
});

```

```
});

// CORS
import cors from 'cors';

export const corsMiddleware = cors({
  origin: process.env.FRONTEND_URL || 'http://localhost:5173',
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
  allowedHeaders: ['Content-Type', 'Authorization', 'X-CSRF-Token'],
});
```

Security Checklist Ringkasan

Area	Proteksi
Password	bcrypt/argon2, min 8 karakter, strength validation
JWT	Access 15m, Refresh 7d, rotation, revoke on logout
Session	Redis-based, httpOnly, secure, sameSite, regenerate on login
Rate Limit	Global + per-endpoint + per-user
Brute Force	Progressive delay → lockout after 5 attempts
CSRF	Token-based (kecuali API Bearer)
XSS	Helmet, CSP, sanitize input
CORS	Whitelist specific origin
Audit Log	Semua action penting di-log
RBAC	Role/permission check per endpoint
MFA	TOTP/SMS/WebAuthn untuk sensitive actions
Email	Verify email sebelum login, jangan bocorin existensi user

Latihan

1. **Rate limiter + brute force protection** — Gabungin rate limit per IP + per email di route login. Implementasi progressive delay: attempt 1-3 = no delay, 4-5 = 1 detik delay, >5 = lockout 15 menit.
2. **RBAC dengan permission system** — Bikin 3 role (admin, moderator, user) dengan permission granular. Admin bisa delete apa

aja, moderator bisa edit post, user cuma bisa baca & buat post sendiri. Implementasi middleware permission check.

3. **Audit log dashboard** — Buat middleware audit logging yang nyatet action penting (login, logout, password reset, delete). Tambah route `/api/admin/audit-logs` dengan pagination + filter.
4. **CSRF + secure cookie setup** — Implementasi CSRF protection di form-based endpoints (bukan API Bearer). Set cookie dengan `httpOnly`, `secure`, `sameSite strict`. Test dengan dan tanpa token.